



Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Технології розроблення програмного забезпечення
Лабораторна робота №3
«Діаграма розгортання. Діаграма компонентів.
Діаграма взаємодій та послідовностей»
Варіант - 4

Виконала:
Студентка групи ІА-22
Третякова Ельвіра

Перевірив:
Мягкий Михайло Юрійович

Київ 2024

ЗМІСТ

Короткі теоретичні відомості	2
<i>Діаграма розгортання</i>	<i>2</i>
<i>Діаграма компонентів</i>	<i>3</i>
<i>Діаграма послідовностей</i>	<i>4</i>
Реалізація частини функціональності системи.....	5
Діаграма послідовностей	8
Діаграма розгортання	9
Діаграма компонентів	11
Висновок	12

Тема: Діаграма розгортання. Діаграма компонентів. Діаграма взаємодій та послідовностей.

Мета: Проаналізувати тему, розробити діаграму розгортання, діаграму компонентів, діаграму взаємодій та послідовностей.

Хід роботи

Варіант – 4: Графічний редактор (proxy, prototype, decorator, bridge, flyweight, SOA)
Графічний редактор повинен вміти створювати / редагувати растрові (або векторні на розсуд студента) зображення в 2-3 основних популярних форматах (bmp, png, jpg), мати панель інструментів для створення графічних примітивів, вибору кольорів, нанесення тексту, додавання найпростіших візуальних ефектів (ч/б растр, інфрачервоний растр, 2-3 на вибір учня), роботи з шарами.

Короткі теоретичні відомості

Діаграма розгортання

Діаграма розгортання — діаграма на якій відображаються обчислювальні вузли під час роботи програми, компоненти, та об'єкти, що виконуються на цих вузлах. Компоненти відповідають представленню робочих екземплярів одиниць коду. Компоненти, що не мають представлення під час роботи програми на таких діаграмах не відображаються; натомість, їх можна відобразити на діаграмах компонентів. Діаграма розгортання відображає робочі екземпляри компонентів, а діаграма компонентів, натомість, відображає зв'язки між типами компонентів.

Діаграма розгортання в UML моделює фізичне розгортання артефактів на вузлах.

Вузли представляються, як прямокутні паралелепіпеди з артефактами, розташованими в них, зображеними у вигляді прямокутників. Вузли можуть мати підвузли, які представляються, як вкладені прямокутні паралелепіпеди. Один вузол діаграми розгортання може концептуально представляти безліч фізичних вузлів, таких, як кластер серверів баз даних.

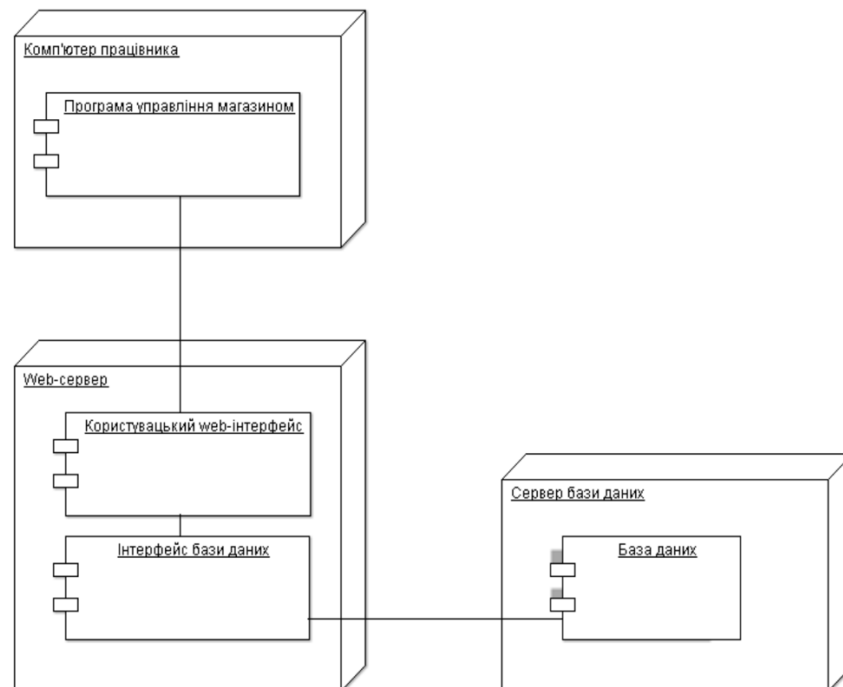
Існує два типи вузлів:

- Вузол пристрою;
- Вузол середовища виконання.

Вузли пристроїв - це фізичні обчислювальні ресурси зі своєю пам'яттю і сервісами для виконання програмного забезпечення, такі як звичайні ПК, мобільні телефони.

Вузол середовища виконання - це програмний обчислювальний ресурс, який працює всередині зовнішнього вузла і який надає собою сервіс, який виконує інші виконувані програмні елементи.

Діаграму розгортання системи можна описати наступним чином. Працівник взаємодіє із системою шляхом використання програми управління магазином. Програма відправляє запити та отримує відповіді від web-сервера. Сервер надає користувацький інтерфейс, а також – використовує інтерфейс бази даних для доступу до даних, їх зміни, видалення, модифікації.



Діаграма компонентів

Діаграма компонентів UML відображає систему, розбиту на окремі модулі.

Виділяють три типи діаграм:

1. Логічні
2. Фізичні
3. Виконувані

Найчастіше використовують логічне розбиття — система представлена як набір незалежних компонентів, що взаємодіють між собою. Наприклад, система продажу продуктів може мати такі компоненти: каса, сервер продаж, система обліку тощо. Компоненти можуть бути розміщені на різних фізичних пристроях або в різних процесах.

Фізичне розбиття фокусується на розподілі компонент між серверами та комп'ютерами, що частіше відображається на діаграмі розгортання.

Виконуване розбиття показує компоненти як файли (наприклад, .exe, html, бази даних), даючи інший ракурс системи. Однак цей підхід не має широкого використання через наявність діаграм розгортання.

Компоненти мають забезпечувати гнучкість для клієнтів, дозволяючи оновлювати систему частинами та забезпечувати сумісність між різними виробниками, хоча це важко реалізувати.

Діаграма послідовностей

Діаграма послідовності — відображає взаємодії об'єктів впорядкованих за часом. Зокрема, такі діаграми відображають задіяні об'єкти та послідовність відправлених повідомлень.

Іншими словами, діаграма послідовностей відображає часові особливості передачі і прийому повідомлень об'єктами.

Діаграми послідовностей можна використовувати для уточнення діаграм прецедентів, більш детального опису логіки сценаріїв використання. Це відмінний засіб документування проекту з точки зору сценаріїв використання. Діаграми послідовностей зазвичай містять об'єкти, які взаємодіють у рамках сценарію, повідомлення, якими вони обмінюються, і які повертаються результати, які пов'язані з повідомленнями.

На діаграмі послідовності зображаються тільки ті об'єкти, які безпосередньо беруть участь у взаємодії.

Лінія життя об'єкта зображується пунктирною вертикальною лінією, асоційованою з єдиним об'єктом на діаграмі послідовності. Лінія життя служить для позначення періоду часу, протягом якого об'єкт існує в системі і, отже, може потенційно брати участь у всіх її взаємодіях. Якщо об'єкт існує в системі постійно, то і його лінія життя повинна продовжуватися по всій площині діаграми послідовності від самої верхньої її частини до самої нижньої.

У процесі функціонування об'єктно-орієнтованих систем одні об'єкти можуть перебувати в активному стані, безпосередньо виконуючи певні дії, або стані пасивного очікування

повідомлень від інших об'єктів. Щоб явно виділити подібну активність об'єктів, в мові UML застосовується спеціальне поняття, що отримало назву фокуса управління. Фокус управління зображується у формі витягнутого вузького прямокутника, верхня сторона якого позначає початок отримання фокусу управління об'єкта, а його нижня сторона - закінчення фокусу управління. Прямокутник розташовується нижче позначення відповідного об'єкта і може замінювати його лінію життя, якщо на всьому її протязі він є активним.

В UML кожна взаємодія описується сукупністю повідомлень, якими ті об'єкти, що беруть участь у ньому обмінюються між собою. Повідомлення є закінченим фрагментом інформації, який відправляється одним об'єктом іншому. Прийом повідомлення ініціює виконання певних дій, спрямованих на вирішення окремого завдання тим об'єктом, якому це повідомлення відправлено.

Реалізація частини функціональності системи

Структуру проєкту після реалізації частини функціональності системи зображена на рис.1.

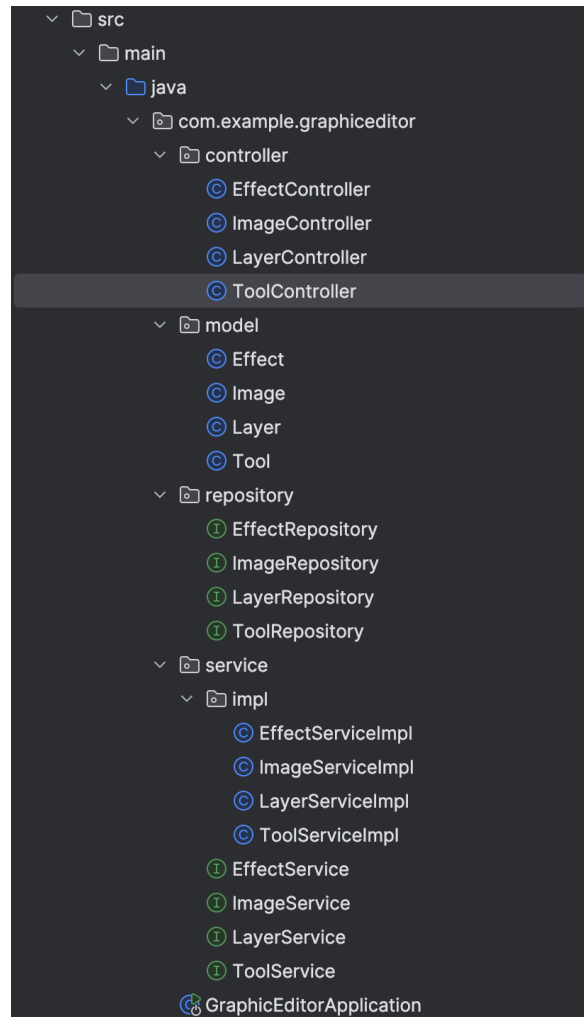


Рис.1

Основні пакети структури проєкту:

1. Пакет контролерів:

- EffectController: Відповідає за обробку запитів, пов'язаних з ефектами графічного редактора. Містить методи для створення нового ефекту, отримання ефекту за його ID, отримання всіх доступних ефектів, оновлення ефекту та його видалення.

- ImageController: Обробляє запити, пов'язані із зображеннями. Забезпечує функціональність для створення нового зображення, отримання конкретного зображення за ID, отримання списку всіх зображень, редагування інформації про зображення та видалення зображення.

- LayerController: Відповідає за обробку запитів, що стосуються шарів в графічному редакторі. Містить методи для створення нового шару, отримання шару за ID, отримання списку всіх шарів, оновлення шару та його видалення.

- ToolController: Обробляє запити, пов'язані з інструментами графічного редактора. Надає можливість створювати новий інструмент, отримувати конкретний інструмент за ID, отримувати всі інструменти, редагувати інструмент та видаляти його.

Кожен з контролерів відповідає за певний аспект графічного редактора, дозволяючи користувачам створювати, отримувати, оновлювати та видаляти відповідні елементи (ефекти, зображення, шари та інструменти) через REST API.

2. Пакет моделей:

- Effect: Модель, що представляє ефект в графічному редакторі. Містить такі атрибути, як “id” для унікальної ідентифікації ефекту та “name” для зберігання назви ефекту.

- Image: Модель, що описує зображення. Включає атрибути “id” для унікальної ідентифікації зображення, “name” для назви, “format” для зберігання формату файлу (наприклад, bmp, png, jpg) та “data”, що містить двійкові дані самого зображення.

- Layer: Модель, що представляє шар зображення. Містить атрибути “id”, “name” для назви шару, “zIndex” для визначення його порядку відображення (шарування) та “visible” для позначення видимості шару.

- Tool: Модель інструменту графічного редактора. Включає “id”, “name” для назви інструменту, а також “icon” для зберігання іконки інструменту.

3. Пакет репозиторіїв:

- EffectRepository: Інтерфейс для роботи з даними про ефекти. Забезпечує методи для зберігання, пошуку та видалення ефектів в базі даних.

- ImageRepository: Інтерфейс для роботи з даними про зображення. Дозволяє зберігати, знаходити та видаляти записи про зображення.

- LayerRepository: Інтерфейс для управління даними про шари. Надає можливість зберігати, знаходити та видаляти інформацію про шари.

- ToolRepository: Інтерфейс для роботи з даними інструментів. Забезпечує функції для зберігання, пошуку та видалення інструментів.

4. Пакет сервісів:

- EffectService: Інтерфейс, що містить бізнес-логіку для обробки ефектів. Забезпечує методи для створення, отримання, оновлення та видалення ефектів, а також для отримання всіх ефектів в системі.

- ImageService: Інтерфейс, що містить бізнес-логіку для роботи з зображеннями. Включає методи для створення нового зображення, отримання зображення за його ID, оновлення зображення, видалення та отримання списку всіх зображень.

- LayerService: Інтерфейс, що містить бізнес-логіку для роботи з шарами. Забезпечує можливість створювати нові шари, отримувати інформацію про шари за ID, оновлювати, видаляти шари та отримувати всі доступні шари.

- ToolService: Інтерфейс, що містить бізнес-логіку для управління інструментами. Включає методи для створення інструменту, отримання інструменту за ID, оновлення та видалення інструменту, а також для отримання списку всіх інструментів.

5. Пакет імплементацій:

- EffectServiceImpl: Клас, що реалізує логіку сервісу для обробки ефектів. Використовує “EffectRepository” для доступу до бази даних і реалізує методи, визначені в інтерфейсі “EffectService”, зокрема для створення, пошуку, оновлення та видалення ефектів.

- ImageServiceImpl: Реалізація сервісу для роботи із зображеннями. Використовує “ImageRepository” для взаємодії з базою даних, забезпечуючи функціонал для створення, отримання, оновлення та видалення зображень, відповідно до інтерфейсу “ImageService”.

- LayerServiceImpl: Клас, що реалізує логіку управління шарами. Використовує “LayerRepository” для доступу до даних про шари і реалізує методи для створення, пошуку, оновлення та видалення шарів, що визначені в “LayerService”.

- ToolServiceImpl: Реалізація сервісу для управління інструментами. Забезпечує виконання методів інтерфейсу “ToolService” через “ToolRepository”, реалізуючи операції створення, пошуку, оновлення та видалення інструментів.

Діаграма послідовностей

Діаграма послідовностей для додавання створення ефекту та отримання всіх ефектів зображена на рис.2.

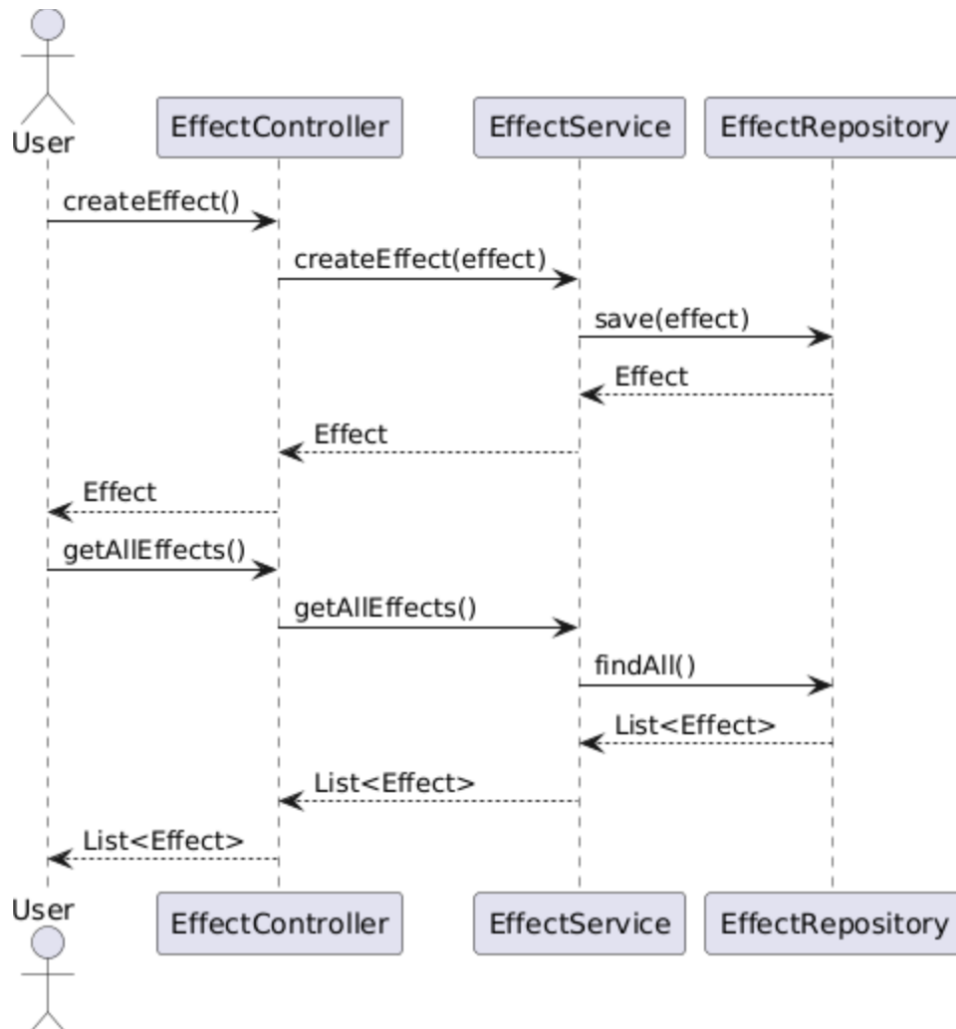


Рис.2

Сценарій створення ефекту:

1. Користувач надсилає запит на створення ефекту до EffectController (EC).
2. EffectController (EC) передає запит до EffectService (ES) для збереження нового ефекту.
3. EffectService (ES) викликає EffectRepository (ER) для збереження ефекту в базі даних.
4. EffectRepository (ER) підтверджує успішне збереження ефекту.
5. EffectService (ES) інформує EffectController (EC) про успішне збереження.

6. EffectController (EC) повертає підтвердження користувачу про створення ефекту.

Сценарій отримання всіх ефектів:

1. Користувач надсилає запит на отримання всіх ефектів до EffectController (EC).
2. EffectController (EC) передає запит до EffectService (ES) для отримання списку ефектів.
3. EffectService (ES) запитує EffectRepository (ER) для пошуку всіх збережених ефектів.
4. EffectRepository (ER) повертає список ефектів.
5. EffectService (ES) передає список ефектів назад до EffectController (EC).
6. EffectController (EC) повертає список ефектів користувачу для відображення.

Ці сценарії показують, як система обробляє запити користувача на створення та отримання ефектів у графічному редакторі.

Діаграма розгортання

Діаграма розгортання зображена на рис.3

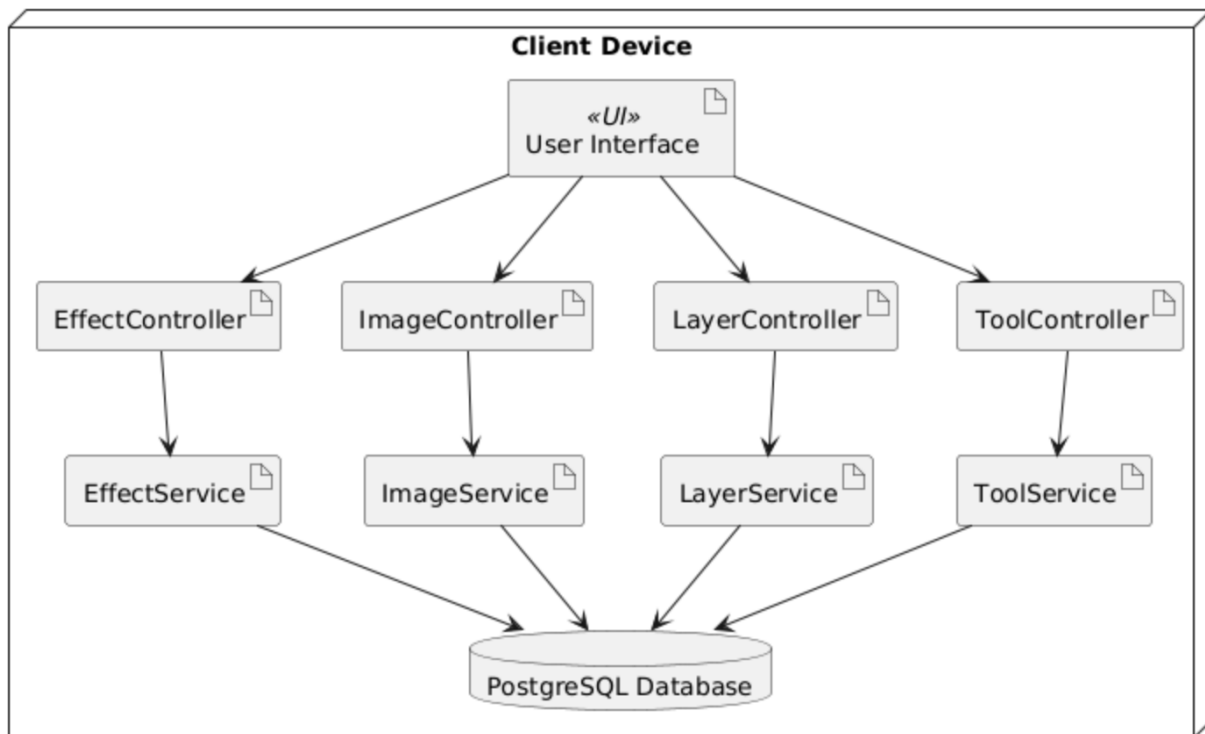


Рис.3

Елементи для діаграми розгортання:

1. Вузли:

Клієнтський пристрій: фізичний комп'ютер користувача, на якому працює весь графічний редактор, включаючи інтерфейс користувача, контролери, сервіси та базу даних.

2. Артефакти:

Інтерфейс користувача: основний фронтенд-додаток графічного редактора.

Контролери та сервіси: серверна логіка додатка, яка обробляє запити від інтерфейсу користувача і виконує операції з даними.

База даних PostgreSQL: зберігає дані для графічного редактора і розгорнута локально.

3. Зв'язки між компонентами:

Локальні виклики між інтерфейсом користувача, контролерами та сервісами.

Локальне підключення до PostgreSQL для взаємодії з базою даних.

Опис Елементів:

Client Device: клієнтський пристрій користувача, на якому запущений весь графічний редактор. Тут розміщені:

Інтерфейс користувача (User Interface).

Контролери та сервіси для обробки логіки програми.

Локально запущена база даних PostgreSQL для зберігання даних програми.

У цій діаграмі розгортання все розміщено на одному вузлі Client Device (клієнтський пристрій), який представляє комп'ютер користувача, де розміщено як інтерфейс користувача, так і всі контролери, сервіси та базу даних. Зв'язки між компонентами є локальними, оскільки все працює в одному середовищі.

Діаграма компонентів

Діаграма компонентів зображена на рис. 4.

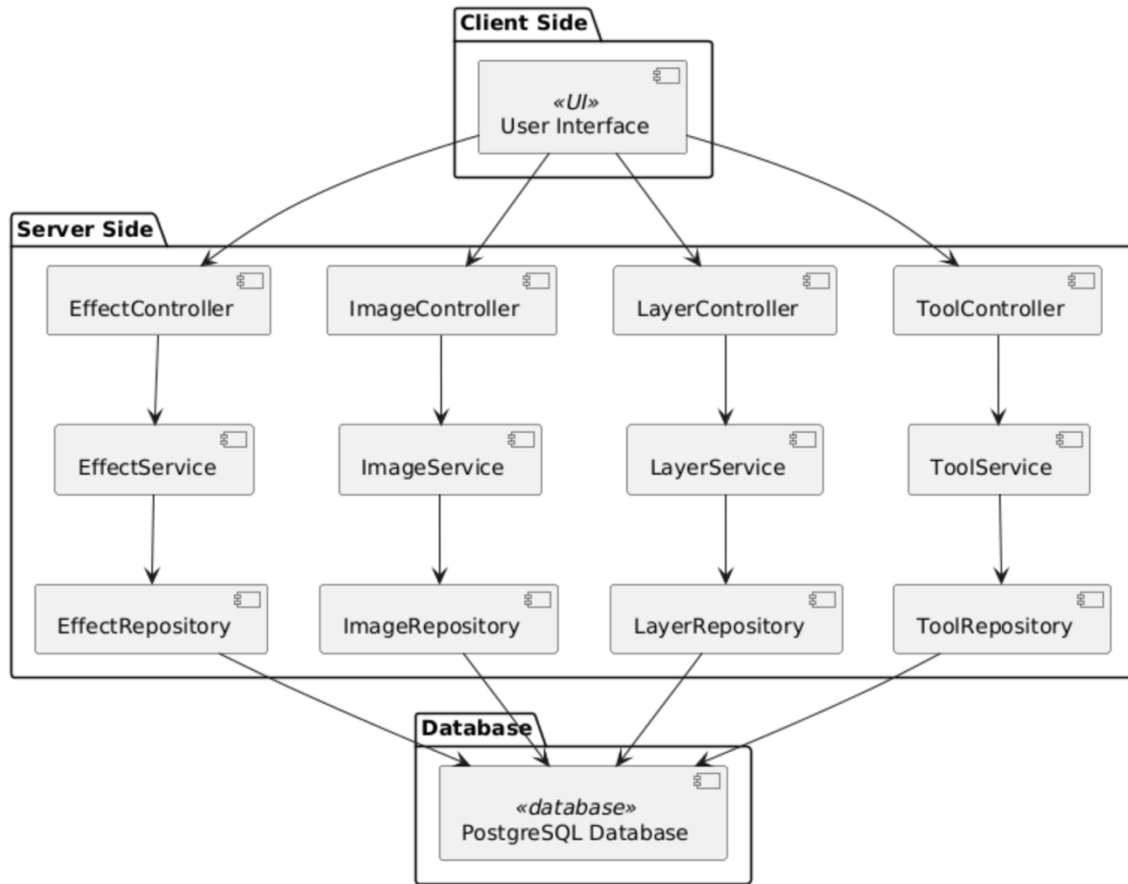


Рис.4

1. Client Side:

- **User Interface:** Цей компонент відповідає за взаємодію користувача з програмою. Він надає інтерфейс для створення та редагування зображень, роботи з шарами, вибору інструментів і застосування ефектів. Інтерфейс користувача напряму звертається до відповідних контролерів на стороні сервера без використання мережевих протоколів, що відповідає моделі настільного додатку.

2. Server Side:

- **EffectController, ImageController, LayerController, ToolController:** Ці контролери приймають запити від інтерфейсу користувача і передають їх до відповідних сервісів. Наприклад, **EffectController** обробляє запити, пов'язані з ефектами, такі як створення нового ефекту або застосування його до зображення.

- **EffectService, ImageService, LayerService, ToolService:** Ці сервіси містять бізнес-логіку для обробки запитів. Вони працюють з відповідними репозиторіями для виконання

операцій, таких як збереження, отримання, оновлення та видалення елементів (ефектів, зображень, шарів і інструментів).

- EffectRepository, ImageRepository, LayerRepository, ToolRepository: Репозиторії виконують доступ до бази даних і містять методи для зберігання та отримання даних для кожного з елементів. Вони взаємодіють із PostgreSQL для забезпечення збереження інформації про ефекти, зображення, шари та інструменти.

3. Database:

- PostgreSQL Database: Використовується як сховище даних для вашого графічного редактора. База даних містить інформацію про всі елементи, такі як ефекти, зображення, шари та інструменти, і забезпечує збереження та отримання даних через відповідні репозиторії.

Ця діаграма ілюструє взаємодію компонентів в межах одного настільного додатку. Інтерфейс користувача звертається до контролерів напряму, а контролери взаємодіють з бізнес-логікою через сервіси та зберігають дані в базі через репозиторії.

Висновок

В даній лабораторній роботі я розробила діаграму розгортання, діаграму компонентів, діаграму послідовностей.